Athletes ETL Pipeline

Moving data from CSV to SQL Server using Python Pandas

Project Overview Athletes ETL Pipeline

Objectives

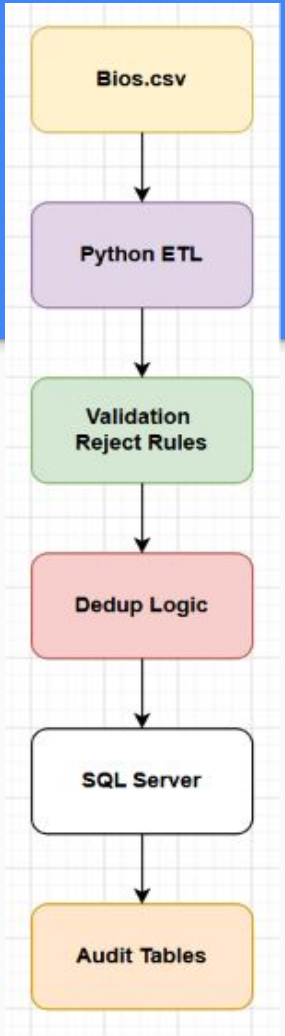
- Read athlete data from CSV
- Clean and validate records
- Reject bad records
- Remove duplicates
- Load into SQL Server
- Track execution metrics
- Orchestrate with Airflow

Diagram

bios.csv -> Python ETL -> Validation Layer -> Dedupe Layer ->

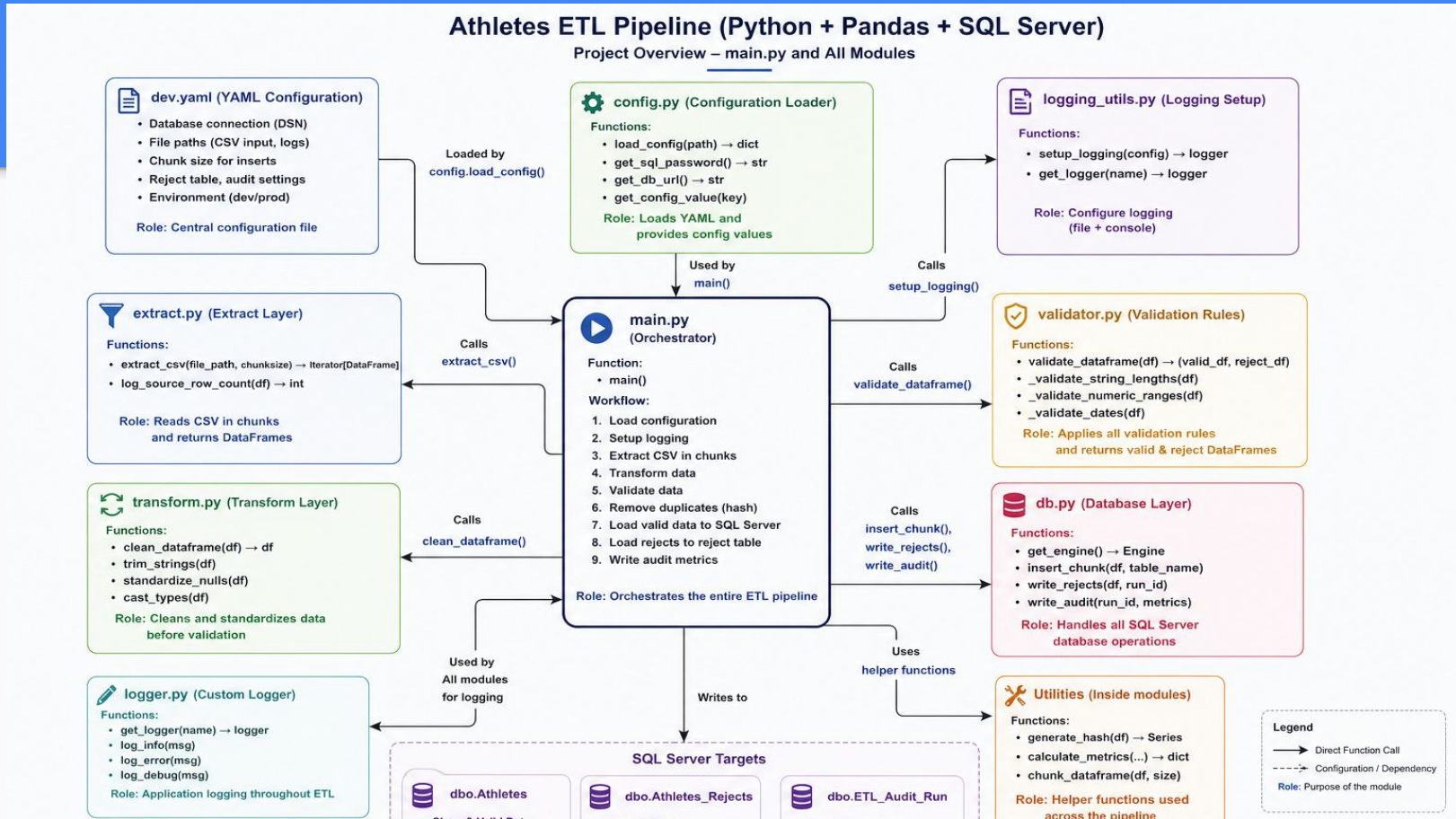
SQL Server -> Audit Tables

High-Level Architecture



Architecture Overview: Application Architecture – Athletes ETL Pipeline

Purpose: Show how the project is organized



Execution Flow (the Story)

This is the slide that explains what happens when someone runs: python -m athletes_etl.main

Step 1

main.py starts

Responsibilities

- Entry point
- Starts ETL
- Coordinates all modules

Calls:

load_config()

Execution Flow (the Story) part 2

Step 2 config.py

Loads dev.yaml

Returns config dictionary

Containing

- SQL Server connection
- CSV location
- Chunk size
- Audit settings
- Reject settings

Returns to main.py

Execution Flow (the Story) part 3

Step 3

logging_utils.py

Creates

- Console logger
- File logger

Returns logger

Execution Flow (the Story) part 4

Step 4

extract.py

Reads	bios.csv
Returns	Pandas DataFrame

Execution Flow (the Story) part 5

Step 5

transform.py

Performs

- Trim strings
- Clean values
- Normalize columns
- Standardize data

Returns Clean DataFrame

Execution Flow (the Story) part 6

Step 6

validator.py

Checks

- String lengths
- Numeric ranges
- Dates
- Nulls

Produces

Valid DataFrame

Reject DataFrame

Execution Flow (the Story) part 7

Step 7

Hash Deduplication

Creates Row Hash

Finds duplicates

Keeps

First occurrence

Removes

Duplicates

Execution Flow (the Story) part 8

Step 8

db.py

Writes

Valid rows	dbo.Athletes
Writes rejects	dbo.Athletes_Rejects
Writes metrics	dbo.ETL_Audit_Run

Execution Flow (the Story) part 9

Step 9

main.py

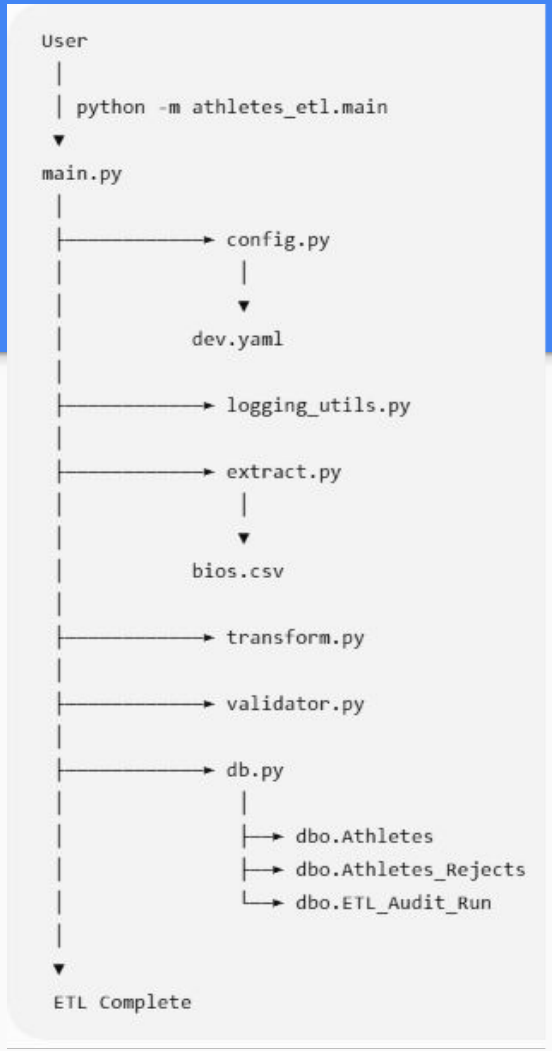
Displays

ETL completed successfully

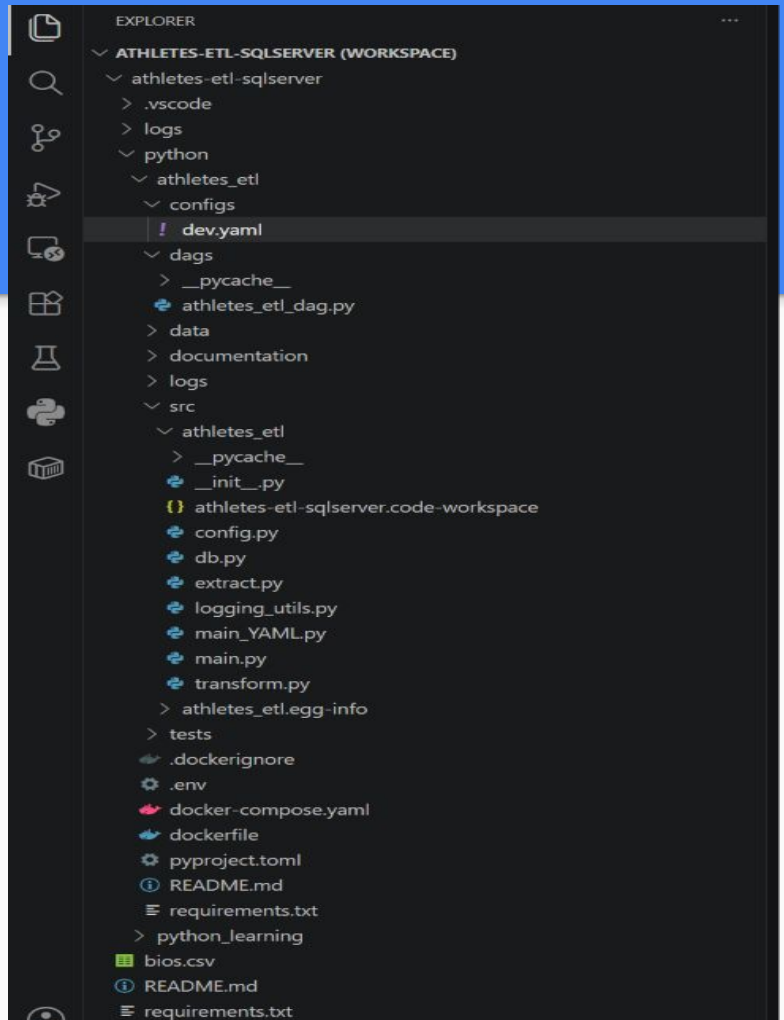
Logs

- Execution time
- Rows loaded
- Rejects
- Duplicates

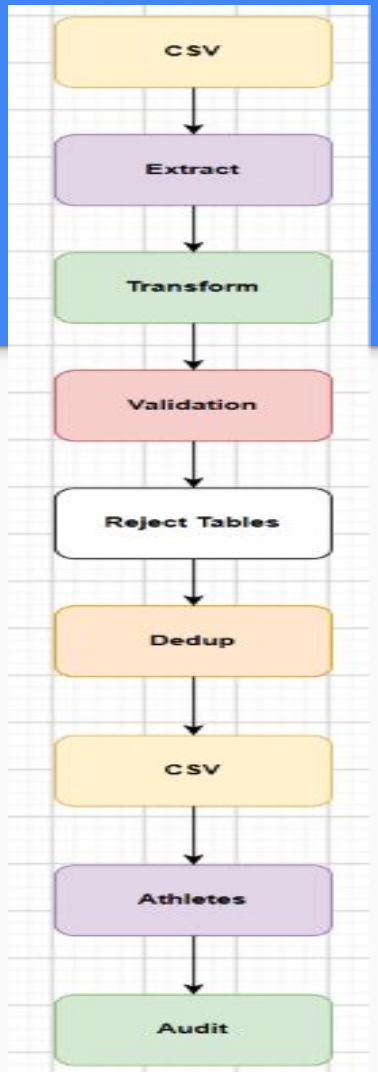
Sequence Diagram (In what order are they executed?)



Folder Structure



Data Flow Diagram

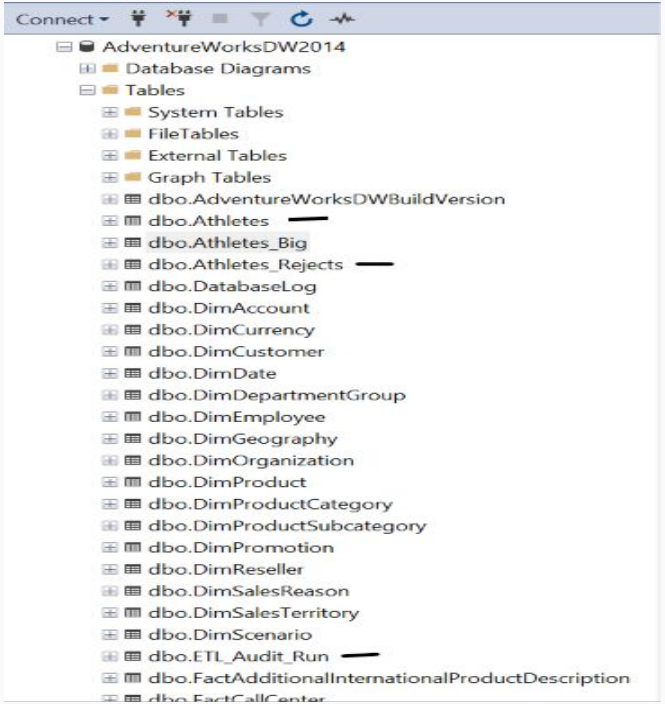


SQL Server Objects

Tables

dbo.Athletes
dbo.Athletes_Rejects
dbo.ETL_Audit_Run

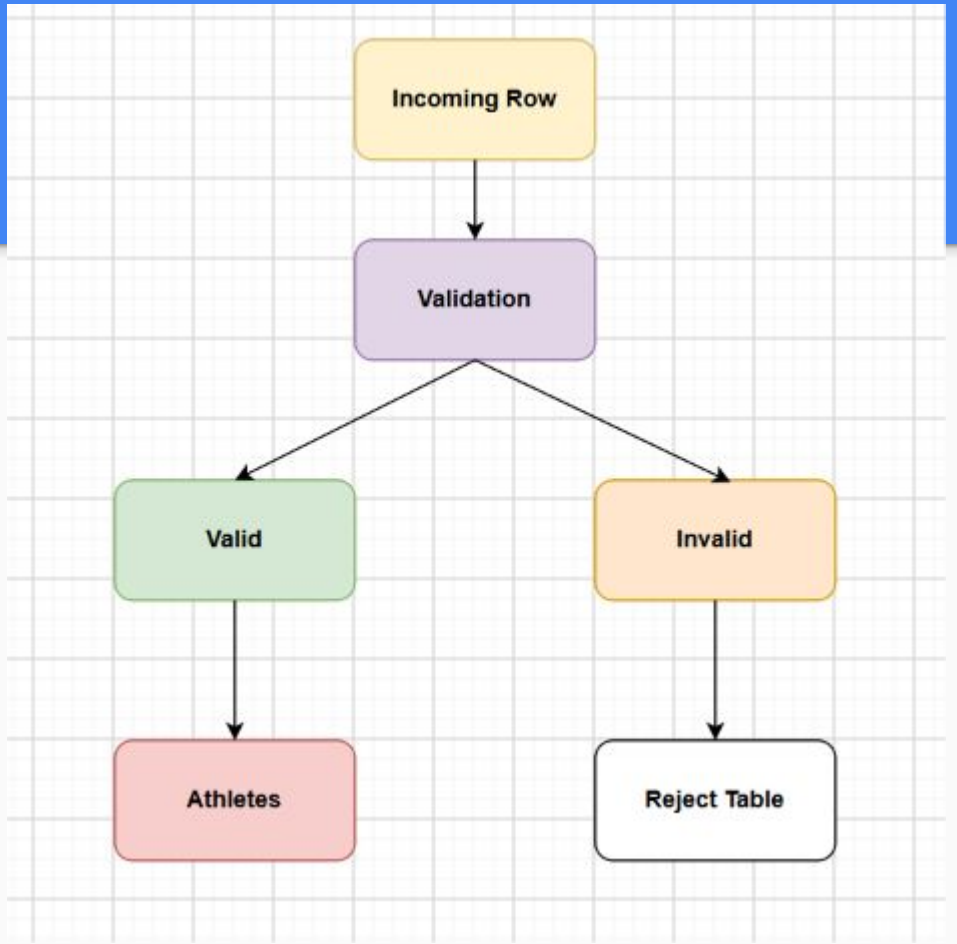
Table	Purpose
Athletes	Final clean data
Athletes_Rejects	Invalid rows
ETL_Audit_Run	ETL metrics



Validation Rules

- String Length Checks
  - name <= 200
  - born_city <= 100
  - born_region <= 100
  - NOC <= 50
- Numeric Checks
  - height_cm 90-250
  - weight_kg 20-300
- Date Checks
  - born_date > 1800
  - born_date <= today
  - died_date > born_date

Reject Handling



Deduplication Logic

- Current Logic:
  - Hash Columns
  -
  - name
  - born_date
  - born_city
  - born_country
- Process:
- Results:
  - 411 duplicates removed

Audit Framework

- **Captured Metrics:**
  - Run ID
  - Start Time
  - End Time
  - Duration
  - Status
  - Rows Loaded
  - Rows Rejected
  - Rows Skipped
- **Diagram:**

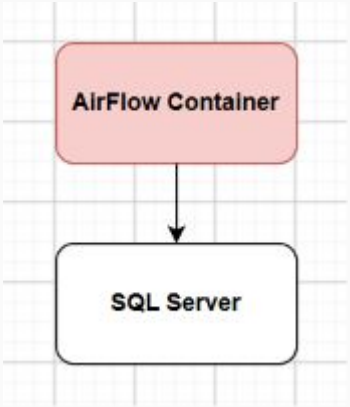
  ETL Run

  ↓

  Audit Table

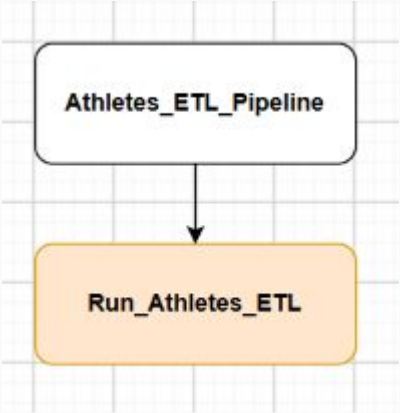
Docker Architecture

- Diagram:
- Volumes:
  - dags
  - src
  - configs
  - data



Airflow DAG

- Diagram:
- Execution:
  - python -m athletes_etl.main (Bash command)



Operational Runbook

- Start
  - docker compose up -d (Bash)
- Stop
  - docker compose down (Bash)
- Check DAGs
  - docker exec -it airflow airflow dags list (Bash)
- Trigger DAGs
  - docker exec -it airflow airflow dags trigger athletes_etl_pipeline (Bash)

Common Issues (Gotchas)

- SQL Password Missing
  - Error:
    - Missing environment variable:
    - SQLSERVER_PASSWORD
  - Fix
    - Set SQLSERVER_PASSWORD

- PYTHONPATH Missing
  - Error:
    - No module named athletes_etl
  - Fix:
    - PYTHONPATH=/opt/airflow/src

Common Issues (Gotchas) part 2

- SQL Login Failures
  - Error:
    - Login failed for user etl_user
  - Fix
    - Set SQLSERVER_PASSWORD

- Duplicate Inserts
  - Problem:
    - Multiple DAG runs
  - Fix:
    - Target-side duplicate check

Lessons Learned

- Environment variables matter
- Docker networking can be tricky
- Airflow needs PostgreSQL metadata DB
- Idempotent loads are essential
- Audit logging simplifies support

Future Enhancements

- Project 2 Roadmap:
- Add:
  - Incremental loads
  - Partitioning
  - CDC
  - Data quality dashboard
  - CI/CD deployment

Production Readiness Assessment

"Production Readiness Assessment"

Area	Status
Data Validation	✅
Audit Logging	✅
Reject Processing	✅
Duplicate Prevention	✅
Airflow Orchestration	✅
Docker Deployment	✅
Unit Testing	⚠️ Partial
CI/CD	❌
Monitoring Dashboard	❌
Cloud Deployment	❌

Business Value Delivered

Business Value Delivered

Feature	Business Benefit
Duplicate Prevention	Reduced storage growth
Reject Processing	Reduced ETL failures
Validation Rules	Improved data quality
Audit Logging	Faster troubleshooting
Airflow Automation	Reduced manual effort
Docker Deployment	Faster onboarding

Interview-Friendly Summary

"What value did your project provide?"

Answer:

> The project was designed to reduce operational costs and improve reliability. It prevents duplicate data from being loaded, isolates bad records instead of failing the pipeline, captures audit information for faster troubleshooting, automates execution through Airflow, and standardizes deployment through Docker. These features reduce storage growth, manual intervention, support effort, and onboarding time while improving data quality and operational stability.